



Bangladesh University of Engineering and Technology
Department of Computer Science and Engineering

CSE200: TECHNICAL WRITING AND PRESENTATION

The A* Search Algorithm

Heuristic shortest-path search, its correctness and its cost

Course name: CSE200
Course title: Technical Writing and Presentation
Subsection: A2
Group: 03

Submitted by

Name	Roll
Zarif Ishmam	2305035
Tonmoy Biswas	2305038
Proteek Rosul	2305042

Abstract

The A* search algorithm finds the cheapest route from a source to a destination in a graph whose edges carry costs. It ranks every place it reaches by the cost of the best route found so far plus an estimate of the cost that remains. This report derives A* from Dijkstra's algorithm, which always finds a cheapest route but searches in every direction, and from greedy best-first search, which heads for the destination but may return a longer route. We state the conditions on the estimate under which A* is guaranteed to return a cheapest route, prove this guarantee, analyze the running time, and explain why A* examines fewer places than Dijkstra's algorithm. Experiments on a grid with an obstacle and on a map of Bangladesh confirm the analysis.

Contents

1	Introduction	1
1.1	Pathfinding	1
1.2	Applications	1
1.3	The need for pathfinding algorithms	2
1.4	Requirements of a pathfinding algorithm	2
2	Preliminaries	2
2.1	Graphs and paths	2
2.2	Heuristic functions	3
2.3	Best-first search	3
2.4	The running example	4
3	Dijkstra's algorithm and greedy best-first search	5
3.1	Dijkstra's algorithm	5
3.2	Greedy best-first search	6
3.3	Comparison	8
4	The A* algorithm	8
4.1	Combining the two keys	8
4.2	A geometric view of the key	9
4.3	Pseudocode	9
4.4	A* on the running example	11
5	Heuristic functions	11
5.1	Requirements	11
5.2	Admissible heuristics	12
5.3	Consistent heuristics	12
5.4	Distance heuristics	13
5.5	How a heuristic guides the search	14
5.6	Misleading heuristics	15
6	Correctness	16
6.1	Two facts about the cost so far	17
6.2	Consistent heuristics	17
6.3	Admissible heuristics	19
7	Time and space complexity	20
7.1	Time	20
7.2	Space	21
7.3	Comparison of the three algorithms	21
8	Why A* expands fewer vertices	22
8.1	The vertices that A* expands	22
8.2	Experiment: a grid with a U-shaped wall	23
9	Worked example on a real map: Dhaka to Sylhet	24
9.1	The district graph	25
9.2	Dijkstra's algorithm	25
9.3	Greedy best-first search	26
9.4	A* search	27
9.5	Comparison	28
9.6	Summary of the experiments	29
10	Variants of A*	30
11	Conclusion	30
	References	31

1 Introduction

Finding the cheapest way from one place to another is one of the most common problems in computing. A navigation system planning a drive from Dhaka to Sylhet, a game character walking around a river and a router forwarding a packet across a network all solve it, often on maps with millions of places and with an answer needed in a fraction of a second. Two classical algorithms approach the problem from opposite directions. Dijkstra’s algorithm always finds a cheapest route but searches evenly in every direction, whereas greedy best-first search heads straight for the destination but may settle for a longer route.

The A* search algorithm of Hart, Nilsson and Raphael [1] combines the strengths of both and is the standard method for this problem when an estimate of the remaining cost is available [2]. It ranks every place that it reaches by the cost of the best route found so far plus an estimate of the cost that remains, such as the straight-line distance to the destination. As long as this estimate never exceeds the true cost, A* returns a cheapest route, yet it examines far fewer places than Dijkstra’s algorithm. In this report we derive A* from its two predecessors, prove when it is correct, analyze its running time and memory, and explain why it examines fewer places. We confirm the analysis on a small example, on a grid with an obstacle and on a map of the 64 districts of Bangladesh.

1.1 Pathfinding

We model a map as a graph: places are vertices, allowed moves are edges, and every edge has a nonnegative cost, such as a length or a travel time. A trip begins at a *source* and ends at a *destination*.

Definition 1 (Pathfinding problem). Given a graph whose edges have nonnegative costs, a source vertex s and a destination vertex t , the *pathfinding problem* asks for a path from s to t whose total cost is minimum.

Figure 1 shows two instances. In the road map, the cheapest route from A to B costs 8 although it visits the most places. In the lattice, a building blocks the direct route, and the cheapest route climbs over the roof in 12 moves instead of eight.

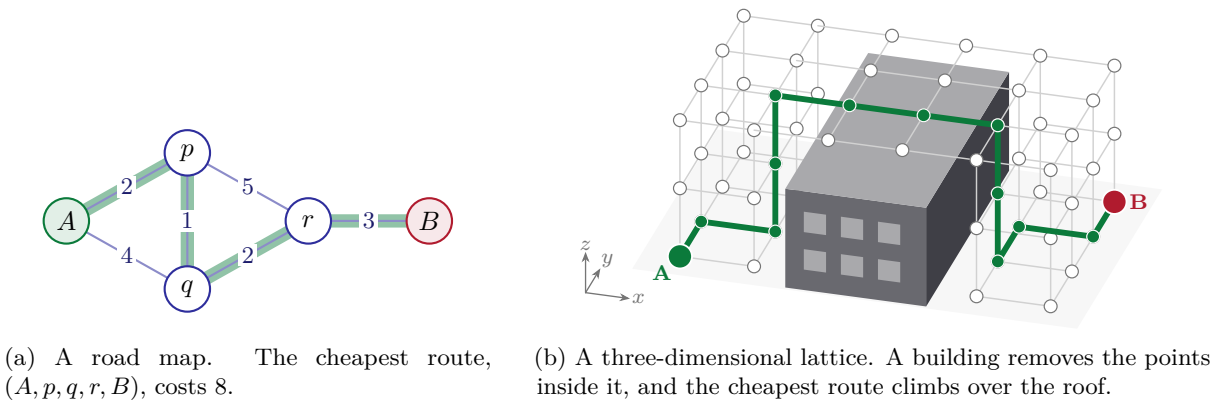


Figure 1: Two instances of the pathfinding problem.

1.2 Applications

Table 1 lists typical applications of pathfinding.

Table 1: Applications of pathfinding.

Application	Vertices	Cost of an edge	Example
Road navigation	road junctions	length or travel time of a road	planning the fastest drive from Dhaka to Sylhet
Video games	tiles of the map	difficulty of the terrain entered	a character that walks around a river to a bridge instead of wading through it
Computer networks	routers	delay or load of a link	the OSPF protocol ¹ , in which every router runs Dijkstra’s algorithm to choose the paths of data packets
Robotics	positions in free space	distance or energy	warehouse robots that carry shelves to packing stations without colliding
Molecular biology	shapes of a protein	energy needed to change shape	the Folding@home project, which searches networks of protein shapes for likely folding pathways
Puzzles	board positions	one per move	the fewest moves that solve the 15-puzzle or Rubik’s Cube [3, 4]

1.3 The need for pathfinding algorithms

Listing every route and keeping the cheapest is infeasible, because the number of routes grows exponentially with the size of the map. On a 50×50 grid there are about 2.5×10^{28} routes between opposite corners that only move right or up; checking a billion per second would take about 800 billion years. Pathfinding algorithms instead grow routes outward from the source and discard a route as soon as a cheaper route to the same place is known.

1.4 Requirements of a pathfinding algorithm

A pathfinding algorithm must serve every pair of source and destination, so we judge it by two requirements.

Optimality. Whenever the destination can be reached from the source, the algorithm returns a cheapest route between them.

Efficiency. The algorithm finds this route while examining as little of the map as possible.

2 Preliminaries

This section fixes the terminology and notation and introduces the running example.

2.1 Graphs and paths

Definition 2 (Weighted graph). A *weighted graph* $G = (V, E, w)$ consists of a finite set V of vertices, a set E of edges, each of which joins two vertices, and a weight function w that assigns a nonnegative cost $w(u, v)$ to every edge (u, v) .

All graphs in this report are undirected. In Figure 2, $V = \{A, p, q, r, B\}$ and $w(A, p) = 2$.

¹OSPF (Open Shortest Path First) is the routing protocol with which the routers inside one network exchange link costs and compute the shortest path to every destination.

Definition 3 (Path and path cost). A *path* from a vertex u to a vertex v is a sequence of vertices $P = (v_0, v_1, \dots, v_k)$ with $v_0 = u$ and $v_k = v$ in which every two consecutive vertices are joined by an edge. The *cost* of the path is

$$c(P) = \sum_{i=1}^k w(v_{i-1}, v_i). \quad (1)$$

For example, the path (A, q, r, B) of Figure 2b costs $4 + 2 + 3 = 9$.

Definition 4 (Distance and shortest path). The *distance* $\delta(u, v)$ from a vertex u to a vertex v is the minimum cost of a path from u to v ; if no such path exists, then $\delta(u, v) = \infty$. A path from u to v of cost $\delta(u, v)$ is a *shortest path* from u to v .

Table 2 lists the paths from A to B that visit no vertex twice; the shortest one has the most edges.

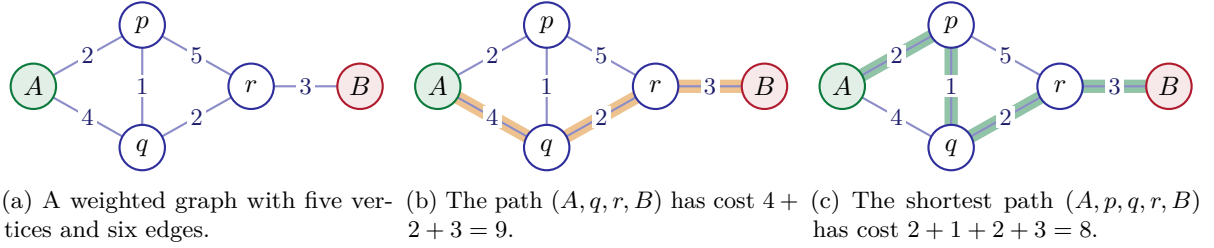


Figure 2: Paths in the road map of Figure 1a.

Table 2: All paths from A to B in Figure 2 that visit no vertex twice.

Path	Edges	Cost	
(A, p, q, r, B)	4	$2 + 1 + 2 + 3 = 8$	shortest, $\delta(A, B) = 8$
(A, q, r, B)	3	$4 + 2 + 3 = 9$	
(A, p, r, B)	3	$2 + 5 + 3 = 10$	
(A, q, p, r, B)	4	$4 + 1 + 5 + 3 = 13$	

2.2 Heuristic functions

A search does not know in advance how far each vertex is from the destination t , but it can use a quick estimate, such as the straight-line distance on a map. We write $h^*(v) = \delta(v, t)$ for the true remaining cost.

Definition 5 (Heuristic function). A *heuristic function*, or simply a *heuristic*, is a function h that assigns to every vertex v a nonnegative number $h(v)$, an estimate of the remaining cost $h^*(v)$.

2.3 Best-first search

All three algorithms of this report are *best-first searches*. They keep a *priority queue* [5] Q of vertices that have been reached but not explored, each with a numerical *key*; a smaller key means a more promising vertex. In each round the search removes a vertex u with the smallest

key and *expands* it: it adds u to the set S of *closed* vertices and examines every edge (u, v) , which may insert v into Q or lower its key. The search stops when it removes the destination. The algorithms differ only in the key.

The pseudocode uses three queue operations: $\text{EXTRACT-MIN}(Q)$ removes and returns a vertex with the smallest key, $\text{INSERT}(Q, v, k)$ adds v with key k , and $\text{INSERT-OR-DECREASE-KEY}(Q, v, k)$ adds v or, if it is already in Q , lowers its key to k . We measure the work of a search by the number of vertices it expands, the destination included. Table 3 collects the notation.

Table 3: Notation used in the report.

Symbol	Meaning
$G = (V, E, w)$	weighted graph with vertex set V , edge set E and weights w
s, t	source vertex and destination vertex
$\delta(u, v)$	distance from u to v , the cost of a shortest path
C^*	distance from s to t , that is, $\delta(s, t)$
$g(v)$	cost of the cheapest path from s to v found so far
$h(v)$	heuristic estimate of the remaining cost from v to t
$h^*(v)$	true remaining cost $\delta(v, t)$
$f(v)$	key used by A*, $f(v) = g(v) + h(v)$
$\pi(v)$	predecessor of v on the cheapest path found so far
Q	priority queue of vertices reached but not yet expanded
S	set of closed vertices, that is, of expanded vertices

2.4 The running example

Figure 3 shows the graph on which we trace the algorithms: eight cities from Dhaka (D) to Sylhet (S), with Gazipur (G), Narsingdi (N), Kishoreganj (K), Brahmanbaria (B), Cumilla (C) and Habiganj (H). The edge weights are small integers chosen for the example, and each disc shows a heuristic value $h(v)$, which never exceeds the true remaining cost (Table 4). The shortest path is (D, N, B, H, S), of cost $C^* = 8$.

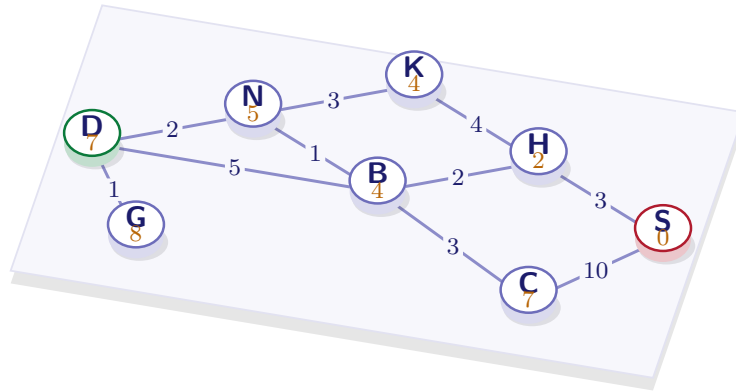


Figure 3: The running example. Each edge shows its weight and each disc the heuristic value h ; the source D has a green rim and the destination S a red rim.

Table 4: The heuristic h of the running example and the true remaining cost h^* .

City	D	G	N	K	B	C	H	S
Heuristic h	7	8	5	4	4	7	2	0
True cost h^*	8	9	6	7	5	8	3	0

3 Dijkstra's algorithm and greedy best-first search

Dijkstra's algorithm [6] ranks a vertex by the cost already paid to reach it, and greedy best-first search ranks it by the estimated cost that remains. Each meets only one of the two requirements of Section 1.4.

3.1 Dijkstra's algorithm

Dijkstra's algorithm always expands the waiting vertex that is cheapest to reach. Its key is $g(v)$, the cost of the cheapest path from the source to v found so far (0 at the source and ∞ elsewhere at the start). Expanding u *relaxes* every edge (u, v) with v not closed: if $g(u) + w(u, v) < g(v)$, the path through u is cheaper, so $g(v)$ is lowered and u becomes the *predecessor* $\pi(v)$. When the destination is extracted, following the predecessors back to the source gives the path. Algorithms 1 and 2 state the procedure in the style of Cormen et al. [7].

Algorithm 1 Dijkstra's algorithm

Input: graph $G = (V, E, w)$ with nonnegative weights, source s , destination t

Output: a shortest path from s to t , or failure

```

1: procedure DIJKSTRA( $G, s, t$ )
2:   for each vertex  $v$  of  $V$  do
3:      $g(v) \leftarrow \infty$ ;  $\pi(v) \leftarrow \text{NIL}$ 
4:   end for
5:    $g(s) \leftarrow 0$ 
6:    $Q \leftarrow$  a priority queue keyed by  $g$  that holds  $s$ 
7:    $S \leftarrow \emptyset$ 
8:   while  $Q$  is not empty do
9:      $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
10:    if  $u = t$  then
11:      return RECONSTRUCT( $\pi, t$ )
12:    end if
13:    add  $u$  to  $S$ 
14:    for each edge  $(u, v)$  such that  $v$  is not in  $S$  do
15:      if  $g(u) + w(u, v) < g(v)$  then
16:         $g(v) \leftarrow g(u) + w(u, v)$ ;  $\pi(v) \leftarrow u$ 
17:        INSERT-OR-DECREASE-KEY( $Q, v, g(v)$ )
18:      end if
19:    end for
20:  end while
21:  return failure  $\triangleright t$  is not reachable from  $s$ 
22: end procedure

```

Table 5 traces Algorithm 1 on the running example; each row lists Q in extraction order (by key, ties first in, first out). The algorithm returns the shortest path of cost 8 but expands all

Algorithm 2 Reconstruction of the path from the predecessors

Input: predecessors π recorded by a search from s , destination t

Output: the path from s to t recorded in π

```
1: procedure RECONSTRUCT( $\pi, t$ )
2:    $P \leftarrow$  an empty list
3:    $v \leftarrow t$ 
4:   while  $v \neq \text{NIL}$  do
5:     insert  $v$  at the front of  $P$ 
6:      $v \leftarrow \pi(v)$  ▷ step back to the predecessor
7:   end while
8:   return  $P$  ▷  $P$  now starts at  $s$  and ends at  $t$ 
9: end procedure
```

eight cities (Figure 4a), including Gazipur, Kishoreganj and Cumilla, which lie off the path, only because they are cheaper to reach than Sylhet.

Table 5: Dijkstra’s algorithm on the running example. The key of a vertex is g .

Step	Q before extraction (vertex: key)	Extracted	Key	Updates
1	D: 0	D	0	$g(N) = 2, g(B) = 5, g(G) = 1$
2	G: 1, N: 2, B: 5	G	1	none
3	N: 2, B: 5	N	2	$g(B) = 3, g(K) = 5$
4	B: 3, K: 5	B	3	$g(H) = 5, g(C) = 6$
5	K: 5, H: 5, C: 6	K	5	none
6	H: 5, C: 6	H	5	$g(S) = 8$
7	C: 6, S: 8	C	6	none
8	S: 8	S	8	destination reached

Remark 1 (Shortcoming of Dijkstra’s algorithm). The key g ignores where the destination lies, so Dijkstra’s algorithm expands every vertex that is cheaper to reach than the destination, in every direction (Proposition 2). It meets the optimality requirement but not the efficiency requirement.

3.2 Greedy best-first search

Greedy best-first search uses the key $h(v)$ and always expands the vertex that looks closest to the destination, whatever it cost to reach. In Algorithm 3 the predecessor of a vertex is fixed when the vertex is first reached, so a route, once found, is never improved.

Table 6 traces it on the running example. It expands only four cities but returns (D, B, H, S) of cost 10 instead of 8 (Figure 4b). In step 2 it prefers B ($h = 4$) to N ($h = 5$), ignoring that the direct road to B costs 5 while the route through N costs 3; since the predecessor of B is already fixed to D, the cheaper route is never found.

Remark 2 (Shortcoming of greedy best-first search). The key h ignores the cost already paid. Greedy best-first search meets the efficiency requirement but not the optimality requirement.

Algorithm 3 Greedy best-first search

Input: graph $G = (V, E, w)$, source s , destination t , heuristic h

Output: a path from s to t , or failure

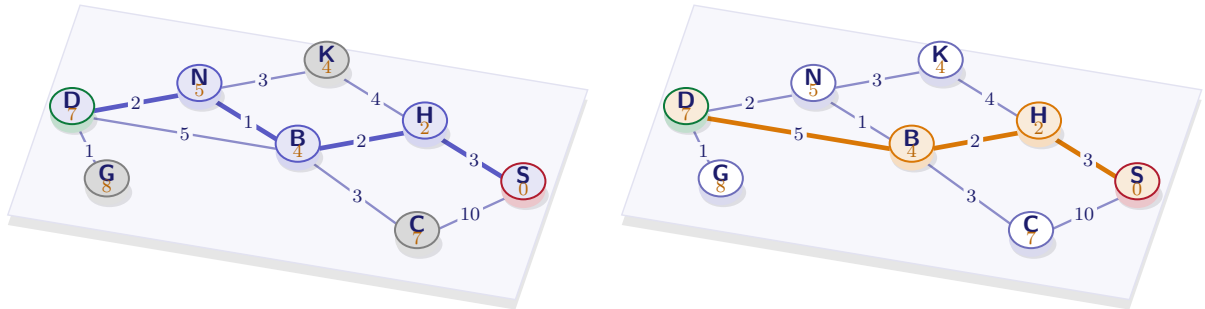
```

1: procedure GREEDY-BEST-FIRST( $G, s, t, h$ )
2:   for each vertex  $v$  of  $V$  do
3:      $\pi(v) \leftarrow \text{NIL}$ 
4:   end for
5:    $R \leftarrow \{s\}$  ▷ vertices reached so far
6:    $Q \leftarrow$  a priority queue keyed by  $h$  that holds  $s$ 
7:   while  $Q$  is not empty do
8:      $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
9:     if  $u = t$  then
10:      return RECONSTRUCT( $\pi, t$ )
11:    end if
12:    for each edge  $(u, v)$  such that  $v$  is not in  $R$  do
13:      add  $v$  to  $R$ ;  $\pi(v) \leftarrow u$ 
14:      INSERT( $Q, v, h(v)$ )
15:    end for
16:  end while
17:  return failure
18: end procedure

```

Table 6: Greedy best-first search on the running example. The key of a vertex is h .

Step	Q before extraction (vertex: key)	Extracted	Key	Vertices reached
1	D: 7	D	7	N, B, G
2	B: 4, N: 5, G: 8	B	4	H, C
3	H: 2, N: 5, C: 7, G: 8	H	2	S, K
4	S: 0, K: 4, N: 5, C: 7, G: 8	S	0	destination reached



(a) Dijkstra's algorithm expands all eight cities and returns a path of cost 8. (b) Greedy best-first search expands four cities and returns a path of cost 10.

Figure 4: The two predecessors of A* on the running example. The thick line is the returned path; gray discs are expanded cities that are not on it.

3.3 Comparison

Table 7 compares the two algorithms; each has the strength that the other lacks.

Table 7: Dijkstra’s algorithm and greedy best-first search compared on the running example.

	Dijkstra’s algorithm	Greedy best-first search
Key of a vertex v	$g(v)$, the cost so far	$h(v)$, the estimated remaining cost
Uses the position of the destination	no	yes
Replaces a found route by a cheaper one	yes	no
Cities expanded	8 of 8	4 of 8
Cost of the returned path	8 (optimal)	10
Optimality requirement	met	not met
Efficiency requirement	not met	met

4 The A* algorithm

A* [1] keeps the cost so far of Dijkstra’s algorithm and adds the estimate of greedy best-first search (Figure 5).

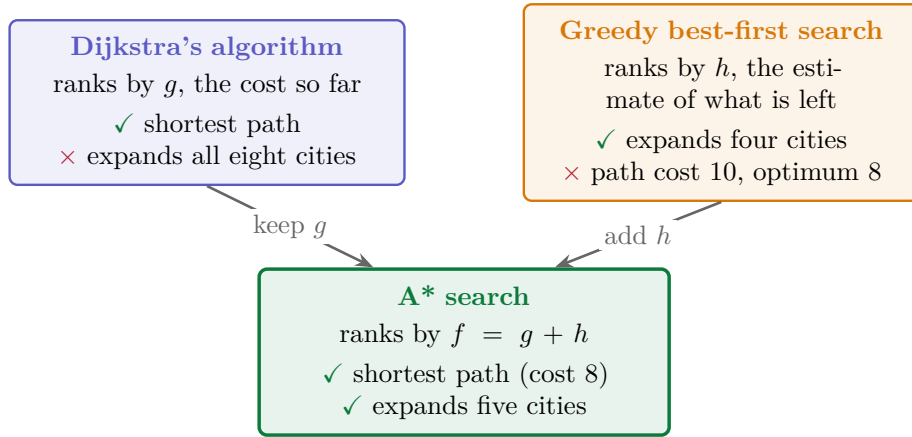


Figure 5: A* takes the cost so far g from Dijkstra’s algorithm and the estimate h from greedy best-first search. The numbers are those of the running example.

4.1 Combining the two keys

A* ranks every vertex v of Q by

$$f(v) = g(v) + h(v), \quad (2)$$

the estimated cost of the cheapest complete route through v (Figure 6). A* thus prefers the vertex on the most promising complete route, rather than the vertex nearest the source or the one that looks nearest the destination (Table 8).

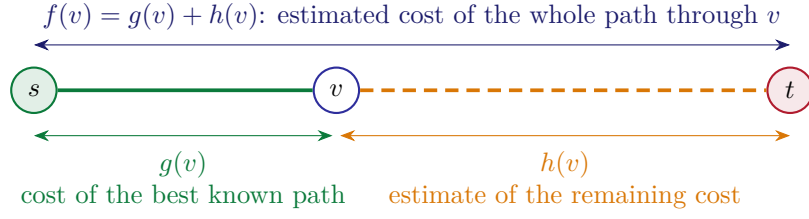


Figure 6: The key of A*. The first part is known exactly; the second part is an estimate.

Table 8: The key of each algorithm.

Algorithm	Key of v	What the key measures
Dijkstra's algorithm	$g(v)$	the cost already paid
Greedy best-first search	$h(v)$	the estimated cost still to pay
A* search	$g(v) + h(v)$	the estimated cost of the whole path

4.2 A geometric view of the key

On an open plane, let g be the distance from s and h the distance to t . Then g and h are cones around s and t , and their sum f is a trough whose floor is the segment from s to t (Figure 7). A* therefore expands the points near this segment first.

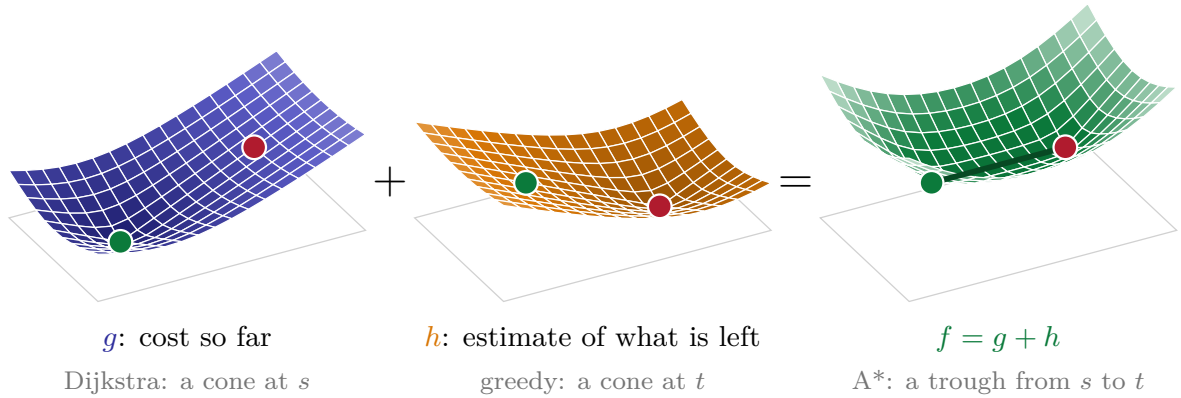


Figure 7: The keys as surfaces over a plane: a cone around s , a cone around t , and their sum, a trough whose floor is the segment from s to t .

4.3 Pseudocode

Algorithm 4 is Algorithm 1 with the key g replaced by $g + h$ in the two lines marked “changed”. With the *zero heuristic*, $h(v) = 0$ for every v , it is exactly Dijkstra’s algorithm. Figure 8 shows the loop shared by the three algorithms.

Algorithm 4 A* search

Input: graph $G = (V, E, w)$ with nonnegative weights, source s , destination t , heuristic h

Output: a path from s to t , or failure

```

1: procedure A-STAR( $G, s, t, h$ )
2:   for each vertex  $v$  of  $V$  do
3:      $g(v) \leftarrow \infty$ ;  $\pi(v) \leftarrow \text{NIL}$ 
4:   end for
5:    $g(s) \leftarrow 0$ 
6:    $Q \leftarrow$  a priority queue keyed by  $g + h$  that holds  $s$  ▷ changed
7:    $S \leftarrow \emptyset$ 
8:   while  $Q$  is not empty do
9:      $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
10:    if  $u = t$  then
11:      return RECONSTRUCT( $\pi, t$ )
12:    end if
13:    add  $u$  to  $S$ 
14:    for each edge  $(u, v)$  such that  $v$  is not in  $S$  do
15:      if  $g(u) + w(u, v) < g(v)$  then
16:         $g(v) \leftarrow g(u) + w(u, v)$ ;  $\pi(v) \leftarrow u$ 
17:        INSERT-OR-DECREASE-KEY( $Q, v, g(v) + h(v)$ ) ▷ changed
18:      end if
19:    end for
20:  end while
21:  return failure
22: end procedure

```

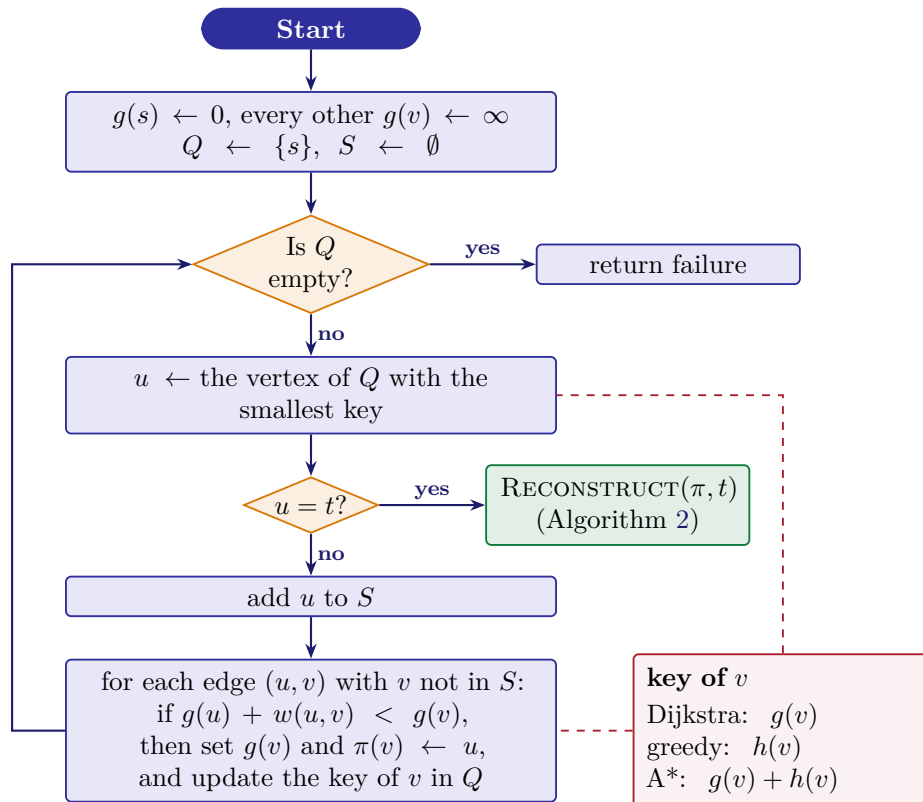


Figure 8: Flow chart of the search loop. The three algorithms differ only in the key of a vertex (red box).

4.4 A* on the running example

Table 9 and Figure 9 show A* on the running example. In step 2, A* expands N (key $2 + 5 = 7$) rather than G (key $1 + 8 = 9$), which Dijkstra’s algorithm would take first. The same step lowers $g(B)$ from 5 to 3 through N, repairing the error of greedy best-first search. In step 5 the destination is extracted with $g = 8$, and G, K and C, whose keys exceed 8, are never expanded. A* thus finds the shortest path with five expansions instead of eight. Sections 5 and 6 show when this result is guaranteed, and Section 8 explains the saving.

Table 9: A* on the running example. The key of a vertex is $f = g + h$.

Step	Q before extraction (vertex: key)	Extracted	$g + h = f$	Updates
1	D: 7	D	$0 + 7 = 7$	$g(N) = 2$, $g(B) = 5$, $g(G) = 1$
2	N: 7, B: 9, G: 9	N	$2 + 5 = 7$	$g(B) = 3$, $g(K) = 5$
3	B: 7, G: 9, K: 9	B	$3 + 4 = 7$	$g(H) = 5$, $g(C) = 6$
4	H: 7, G: 9, K: 9, C: 13	H	$5 + 2 = 7$	$g(S) = 8$
5	S: 8, G: 9, K: 9, C: 13	S	$8 + 0 = 8$	destination reached

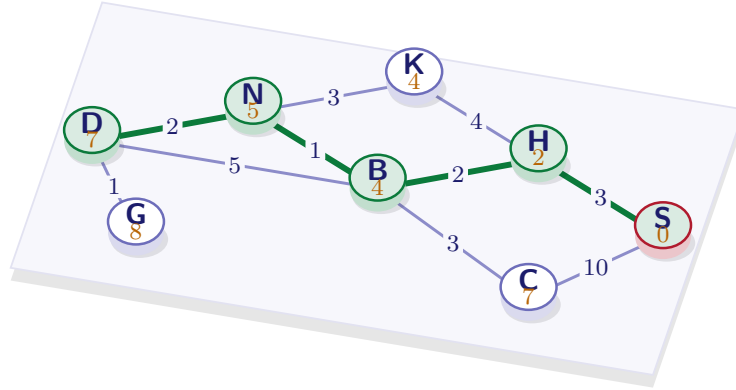


Figure 9: A* on the running example. The green path is the path that A* returns, and its cities are exactly the cities that A* expands.

5 Heuristic functions

The heuristic is the only application-specific part of A*. This section defines the conditions under which A* is correct, presents the distance heuristics of our experiments, and shows how a heuristic guides or misleads the search.

5.1 Requirements

A good heuristic h meets three requirements.

- (i) **It is cheap to compute**, because A* computes it for every vertex it reaches.
- (ii) **It never overestimates**: $h(v) \leq h^*(v)$ for every v . This makes A* return a shortest path (Section 6).
- (iii) **It is close to h^*** , because larger values leave fewer vertices for A* to expand (Section 8).

The best heuristic is therefore as large as possible without exceeding the true cost.

5.2 Admissible heuristics

Requirement (ii) is called admissibility.

Definition 6 (Admissible heuristic). A heuristic h is *admissible* if $0 \leq h(v) \leq h^*(v)$ for every vertex v .

An admissible heuristic is optimistic. The heuristic of the running example is admissible (Table 4); Section 5.6.1 gives one that is not. At the destination an admissible heuristic is zero, since $0 \leq h(t) \leq h^*(t) = 0$.

5.3 Consistent heuristics

Algorithm 4 needs a stronger condition.

Definition 7 (Consistent heuristic). A heuristic h is *consistent* if $h(t) = 0$ and

$$h(u) \leq w(u, v) + h(v) \quad (3)$$

for every edge (u, v) .

In words, one step lowers the estimate by at most the cost of that step, a triangle inequality (Figure 10). On undirected graphs it must hold in both directions of every edge. The heuristic of the running example is consistent; for instance, $h(\mathbf{N}) = 5 = w(\mathbf{N}, \mathbf{B}) + h(\mathbf{B})$.

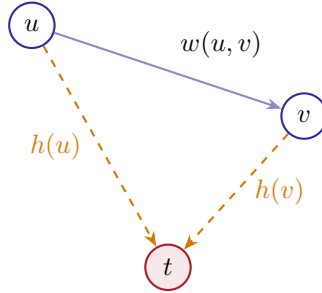


Figure 10: Consistency as a triangle inequality: $h(u) \leq w(u, v) + h(v)$ for every edge (u, v) .

Consistency is the stronger condition.

Lemma 1. *Every consistent heuristic is admissible.*

Proof. Let h be a consistent heuristic. By Definition 5, h takes nonnegative values, so it remains to show that $h(v) \leq h^*(v)$ for every vertex v . If t is not reachable from v , then $h^*(v) = \infty$ and there is nothing to prove. Otherwise let $(v = u_0, u_1, \dots, u_m = t)$ be a shortest path from v to t . We show by induction on $m - i$ that $h(u_i) \leq \delta(u_i, t)$. For $i = m$ we have $h(t) = 0 = \delta(t, t)$. For $i < m$, Inequality (3) and the induction hypothesis give

$$h(u_i) \leq w(u_i, u_{i+1}) + h(u_{i+1}) \leq w(u_i, u_{i+1}) + \delta(u_{i+1}, t) = \delta(u_i, t),$$

where the last equality holds because every part of a shortest path is itself a shortest path. For $i = 0$ this is $h(v) \leq h^*(v)$. $\square$

The converse is false; Example 1 gives an admissible heuristic that is not consistent.

5.4 Distance heuristics

On maps and grids the usual heuristics are distances between the coordinates of a vertex $v = (x_v, y_v)$ and of the destination $t = (x_t, y_t)$.

Definition 8 (Manhattan and Euclidean distance). The *Manhattan distance* and the *Euclidean distance* from v to t are

$$h_M(v) = |x_v - x_t| + |y_v - y_t|, \quad (4)$$

$$h_E(v) = \sqrt{(x_v - x_t)^2 + (y_v - y_t)^2}, \quad (5)$$

respectively. In three dimensions each formula gains a term for the third coordinate.

The Manhattan distance follows the axes, like a car on a street grid; the Euclidean distance is the straight line. For a displacement of $(4, 3, 2)$ they are 9 and $\sqrt{29} \approx 5.39$ (Figure 11). As surfaces around the destination they form a pyramid and a cone (Figure 12), and the pyramid never lies below the cone.

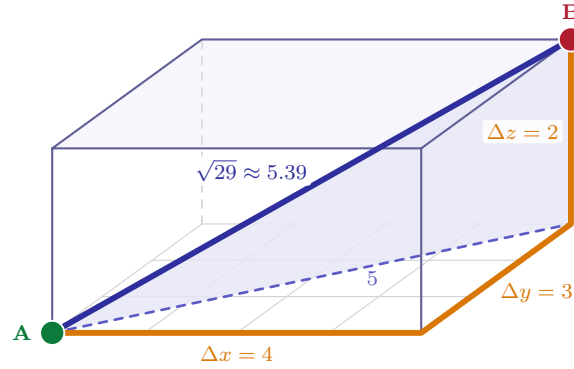


Figure 11: The Manhattan distance (orange, along the edges of the box) and the Euclidean distance (blue, straight through the box) from A to B .

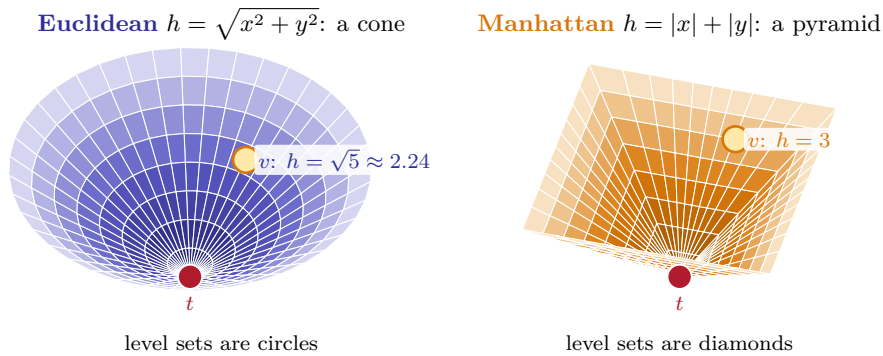


Figure 12: The two distance heuristics as surfaces around the destination t . At the point v with offset $(2, 1)$ from t the pyramid is higher: 3 against $\sqrt{5} \approx 2.24$.

Proposition 1. For any two points v and t of the plane, $h_E(v) \leq h_M(v)$, with equality if and only if v and t agree in at least one coordinate.

Proof. Let $a = |x_v - x_t|$ and $b = |y_v - y_t|$. Both distances are nonnegative, so it suffices to compare their squares. We have $h_M(v)^2 = (a + b)^2 = a^2 + b^2 + 2ab = h_E(v)^2 + 2ab$. Since

$2ab \geq 0$, it follows that $h_E(v) \leq h_M(v)$, with equality exactly when $ab = 0$, that is, when $a = 0$ or $b = 0$. $\square$

Table 10 shows where each distance is admissible. On a unit-cost grid with four neighbors per cell, the Manhattan distance equals the true number of moves when no obstacle intervenes. With straight-line edge weights, no chain of edges is shorter than the straight line, so the straight-line distance is admissible, while the larger Manhattan distance may overestimate.

Table 10: When a distance heuristic never overestimates.

Graph	Admissible heuristic	Reason
Grid with four neighbors per cell, every move of cost 1 (Section 8.2)	Manhattan distance	every move changes one coordinate by 1
Edge weights equal to straight-line distances (Section 9.1)	Euclidean or great-circle distance	no chain of edges is shorter than the direct line
Edge weights unrelated to the coordinates (Section 5.6.1)	neither, in general	a distance can exceed the true cost

In both cases the admissible heuristic is also consistent: neighboring grid cells differ by exactly 1 in Manhattan distance, and a straight line is never longer than a detour through a neighbor.

5.5 How a heuristic guides the search

Lifting every vertex to height $h(v)$ turns the heuristic into a funnel around the destination (Figure 13a). Table 11 shows how the key changes for a unit move on a grid with the Euclidean heuristic (Figure 13b).

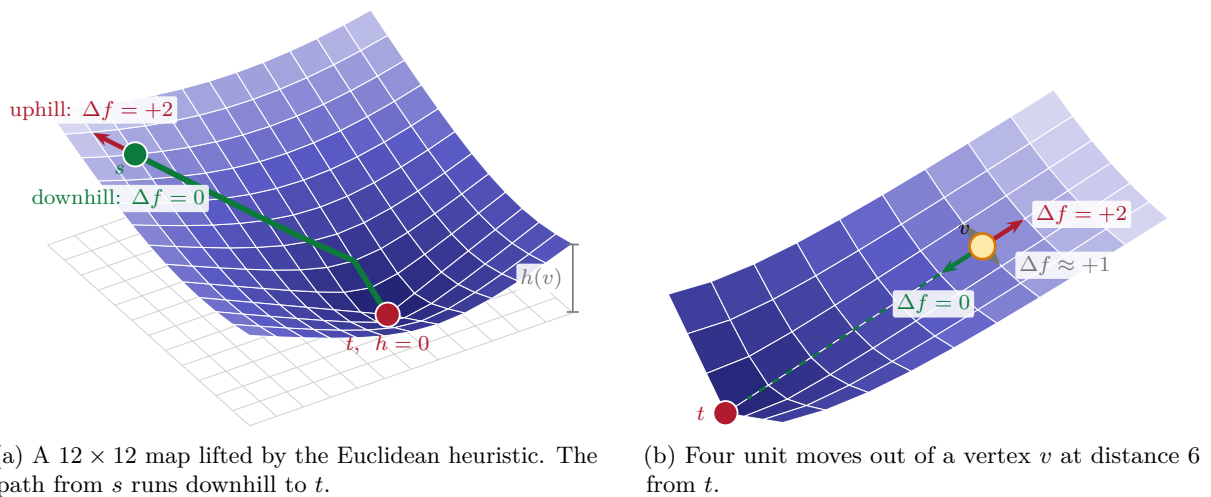


Figure 13: The heuristic as a landscape.

A move toward the destination leaves the key unchanged, while a move away raises it by 2. Such moves are not forbidden, only postponed, so A* expands vertices in the direction of the destination first.

Table 11: Change of the key for a unit move out of a vertex at distance 6 from the destination, with the Euclidean heuristic.

Move	Δg	Δh	Δf
toward the destination	+1	-1	0
away from the destination	+1	+1	+2
sideways	+1	$\sqrt{37} - 6 \approx +0.08$	$\approx +1.08$

5.6 Misleading heuristics

Table 12 lists three ways in which a heuristic can mislead A*. Only an overestimating heuristic can cost the shortest path; the other two only waste work.

Table 12: Three ways in which a heuristic can mislead A*.

Kind	Property	Shortest path	Effect
Overestimating	not admissible	not guaranteed	A* behaves like greedy best-first search; on the running example the returned cost rises from 8 to 10 (Section 5.6.1)
Uninformative	admissible, nearly constant	guaranteed	$h(v) = 1$ for every $v \neq t$ (admissible if every edge costs at least 1) adds the same constant to almost every key, so A* expands almost the same vertices as Dijkstra's algorithm
Blind to obstacles	admissible, ignores walls	guaranteed	A* expands dead ends that appear close to the destination, such as the inside of the U-shaped wall of Section 8.2

5.6.1 An overestimating heuristic

We place the cities of the running example on a grid at $D = (0, 4)$, $N = (4, 6)$, $B = (8, 4)$, $H = (12, 6)$, $S = (16, 4)$, $G = (2, 1)$, $K = (8, 8)$ and $C = (13, 1)$ (Figure 14), and use the Manhattan distance h_M to S as the heuristic. The grid units are unrelated to the edge weights, and h_M overestimates in six of the eight cities (Table 13).

Table 13: Two heuristics for the running example and the true remaining cost. Red values overestimate.

City	D	G	N	K	B	C	H	S
Heuristic h (Table 4)	7	8	5	4	4	7	2	0
Manhattan distance h_M	16	17	14	12	8	6	6	0
True cost h^*	8	9	6	7	5	8	3	0

With h_M , A* returns (D, B, H, S) of cost 10 (Table 14). The cause is $h_M(N) = 14$, more than twice $h^*(N) = 6$: after step 1, N has key 16 and B key 13, so the search reaches S with $g = 10$

before expanding N. With $h(N) = 5$, the key of N would be $7 < 10$. When h dominates g , A^* behaves like greedy best-first search and returns the same path.

Table 14: A^* on the running example with the Manhattan distance h_M , which overestimates.

Step	Q before extraction (vertex: key)	Extracted	$g + h_M = f$	Updates
1	D: 16	D	$0 + 16 = 16$	$g(N) = 2, g(B) = 5, g(G) = 1$
2	B: 13, N: 16, G: 18	B	$5 + 8 = 13$	$g(H) = 7, g(C) = 8$
3	H: 13, C: 14, N: 16, G: 18	H	$7 + 6 = 13$	$g(S) = 10, g(K) = 11$
4	S: 10, C: 14, N: 16, G: 18, K: 23	S	$10 + 0 = 10$	destination reached

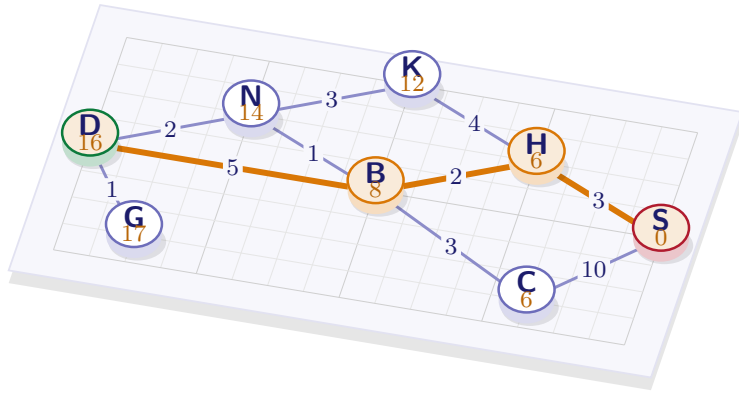


Figure 14: The running example on its grid, with the Manhattan distance h_M in each disc. A^* now returns the orange path, which is not the shortest one.

Figure 15 compares three estimates for N; only the one above the true cost leads A^* astray.

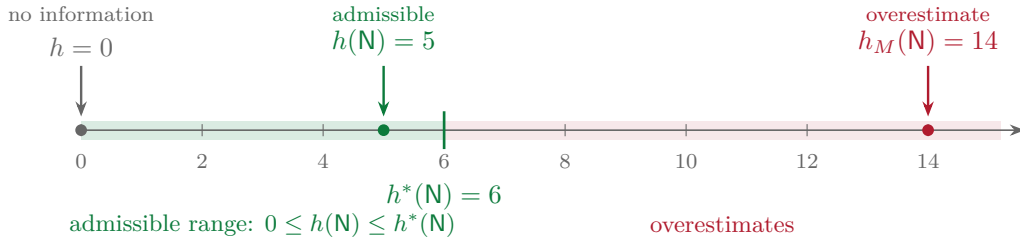


Figure 15: Three estimates of the remaining cost from N, compared with the true cost $h^*(N) = 6$.

6 Correctness

When is the path returned by A^* a cheapest one? Table 15 summarizes the answer, which depends on the heuristic. Throughout this section, $G = (V, E, w)$ is finite with nonnegative weights, t is reachable from s , and $C^* = \delta(s, t)$; the key of a vertex u in Q is $f(u) = g(u) + h(u)$.

Table 15: What A* guarantees for each kind of heuristic.

Heuristic	Algorithm 4 (closed set)	A* with reopening	Result
consistent	shortest path, each vertex expanded at most once	shortest path	Theorem 1
admissible, not consistent	may fail (Example 1)	shortest path	Theorem 2
not admissible	may fail (Section 5.6.1)	may fail	none
$h(v) = 0$ for all v (Dijkstra's algorithm)	shortest path	shortest path	Corollary 1

6.1 Two facts about the cost so far

The values $g(v)$ only go down and never drop below the true distance from the source. The same proof applies to A* with reopening (Section 6.3). A *walk* is like a path but may visit a vertex more than once.

Lemma 2. *During the execution of Algorithm 4, the value $g(v)$ of a vertex v never increases. Whenever $g(v)$ is finite, it is the cost of some walk from s to v , and consequently $g(v) \geq \delta(s, v)$.*

Proof. The algorithm changes $g(v)$ only by the assignment $g(v) \leftarrow g(u) + w(u, v)$, and only when the new value is smaller, so $g(v)$ never increases. We prove the second claim by induction on the number of assignments. Initially only $g(s)$ is finite, and $g(s) = 0$ is the cost of the walk (s) . An assignment sets $g(v) = g(u) + w(u, v)$, where, by the induction hypothesis, $g(u)$ is the cost of a walk from s to u . Appending v to this walk gives a walk from s to v of cost $g(v)$. Finally, removing the repeated parts of a walk yields a path between the same vertices, and the path costs no more than the walk because every weight is nonnegative. Hence $g(v) \geq \delta(s, v)$. $\square$

6.2 Consistent heuristics

With a consistent heuristic, the estimated total cost never decreases along a path.

Lemma 3. *Let h be a consistent heuristic and let $P = (v_0, v_1, \dots, v_k)$ be a path. For $0 \leq i \leq k$, let $f_P(v_i) = c(v_0, \dots, v_i) + h(v_i)$, where $c(v_0, \dots, v_i)$ is the cost of the first i edges of P . Then $f_P(v_0) \leq f_P(v_1) \leq \dots \leq f_P(v_k)$.*

Proof. For $1 \leq i \leq k$, Inequality (3) applied to the edge (v_{i-1}, v_i) gives

$$f_P(v_i) = f_P(v_{i-1}) + w(v_{i-1}, v_i) + h(v_i) - h(v_{i-1}) \geq f_P(v_{i-1}). \quad (6)$$

$\square$

Consequently, the extracted keys never decrease.

Lemma 4. *Let h be a consistent heuristic. Then Algorithm 4 extracts vertices from Q in nondecreasing order of their keys.*

Proof. Suppose that the algorithm extracts a vertex u with key $f(u)$. At that moment every key in Q is at least $f(u)$, because EXTRACT-MIN returns a smallest key. While u is expanded, a key is inserted or decreased only to a value $g(u) + w(u, v) + h(v)$ for a neighbor v of u , and by

Inequality (3) this value is at least $g(u) + h(u) = f(u)$. Hence every key in Q is still at least $f(u)$ at the next extraction, and the next extracted key is at least $f(u)$. $\square$

The main lemma states that a vertex leaves Q only when its value g is exact (Figure 16).

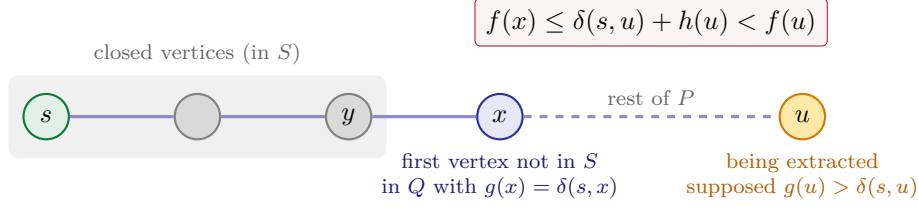


Figure 16: The idea behind Lemma 5. On a shortest path P from s to u , the first vertex x that is not closed is in Q with an exact value $g(x)$, and its key is smaller than the key of u .

Lemma 5. *Let h be a consistent heuristic. Whenever Algorithm 4 extracts a vertex u from Q , we have $g(u) = \delta(s, u)$.*

Proof. Suppose the contrary, and let u be the first extracted vertex with $g(u) \neq \delta(s, u)$; by Lemma 2, $g(u) > \delta(s, u)$. The first iteration extracts s with $g(s) = 0 = \delta(s, s)$, so $u \neq s$. The value $g(u)$ is finite, so u is reachable from s ; let P be a shortest path from s to u . Just before u is extracted, s is in S and u is not. Let x be the first vertex of P that is not in S , and let y be the vertex that precedes x on P ; then y is in S .

The vertex y was extracted before u , so $g(y) = \delta(s, y)$ by the choice of u . When y was expanded, the vertex x was not in S , because S only grows, and the relaxation of the edge (y, x) ensured that $g(x) \leq \delta(s, y) + w(y, x) = \delta(s, x)$; the equality holds because the part of P from s to x is a shortest path. By Lemma 2, $g(x)$ has not increased since and is at least $\delta(s, x)$, so $g(x) = \delta(s, x)$. A vertex with a finite value g that is not in S is in Q , so x is in Q . Moreover $x \neq u$, because $g(u) > \delta(s, u)$.

Lemma 3, applied to P , compares the keys of x and u . The costs of the parts of P that end at x and at u are $\delta(s, x)$ and $\delta(s, u)$, so

$$f(x) = \delta(s, x) + h(x) \leq \delta(s, u) + h(u) < g(u) + h(u) = f(u). \quad (7)$$

The vertex x is in Q with a smaller key than u , which contradicts the choice of u by EXTRACT-MIN. $\square$

We can now prove the main result.

Theorem 1 (Optimality of A* with a consistent heuristic). *Let G be a finite graph with nonnegative edge weights, let s and t be vertices of G such that t is reachable from s , and let h be a consistent heuristic. Then Algorithm 4 expands every vertex at most once and returns a path from s to t of cost $\delta(s, t)$.*

Proof. A vertex enters S when it is extracted and never leaves S , and the loop over the edges never inserts a vertex of S into Q . Hence every vertex is extracted at most once, and the algorithm stops after at most $|V|$ iterations.

The algorithm does not return failure. Suppose, to the contrary, that Q becomes empty before t is extracted. Consider a path from s to t ; its first vertex s is in S and its last vertex t is not. Let x be the first vertex of the path that is not in S . The vertex that precedes x on the path is in S , and its expansion gave x a finite value $g(x)$ and inserted x into Q . The vertex x can leave Q only by being extracted, after which it would be in S or the algorithm would have

returned. Both cases contradict the assumptions, so the algorithm extracts t , and $g(t) = \delta(s, t)$ at that moment by Lemma 5.

It remains to show that the returned path has cost $g(t)$. Let $v \neq s$ be a vertex on the path and let $p = \pi(v)$. The last change of $g(v)$ and $\pi(v)$ happened during the expansion of p and set $g(v) = g(p) + w(p, v)$. The value $g(p)$ did not change afterwards, because p was in S . The vertex p was extracted before v , so following the predecessors from t visits vertices in strictly decreasing order of extraction and therefore reaches s . Adding the equalities $g(v) - g(\pi(v)) = w(\pi(v), v)$ along the path shows that its cost is $g(t) - g(s) = \delta(s, t)$. $\square$

Dijkstra's algorithm is the special case of the zero heuristic.

Corollary 1. *Let G be a finite graph with nonnegative edge weights, and let s and t be vertices of G such that t is reachable from s . Then Algorithm 1 returns a shortest path from s to t .*

Proof. The zero heuristic, $h(v) = 0$ for every vertex v , is consistent, because $0 \leq w(u, v) + 0$ for every edge (u, v) . With this heuristic the key $g + h$ equals g , so Algorithm 4 is Algorithm 1. The claim follows from Theorem 1. $\square$

6.3 Admissible heuristics

Admissibility alone does not make Algorithm 4 correct.

Example 1. Consider the graph of Figure 17 with source s and destination t . The shortest path is (s, a, c, t) , of cost 5. The true remaining costs are $h^*(s) = 5$, $h^*(a) = 4$, $h^*(b) = 5$ and $h^*(c) = 3$, so the heuristic shown in the figure is admissible. It is not consistent, because $h(a) = 4 > w(a, c) + h(c) = 2$. Table 16 traces Algorithm 4 on this graph. In step 4 the edge (a, c) offers c the smaller value $1 + 1 = 2$, but c is already closed, so the algorithm skips the edge and returns (s, b, c, t) , of cost 6.

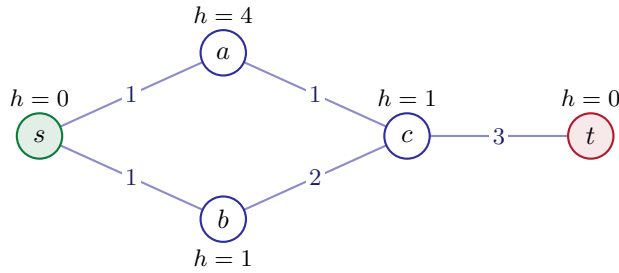


Figure 17: An admissible heuristic that is not consistent. Edge weights are on the edges, and heuristic values are beside the vertices.

Table 16: Algorithm 4 on the graph of Figure 17.

Step	Extracted (key)	Updates	State of c
1	s (0)	$g(a) = 1$, key 5; $g(b) = 1$, key 2	not reached
2	b (2)	$g(c) = 3$, key 4	in Q , $g(c) = 3$
3	c (4)	$g(t) = 6$, key 6	closed, $g(c) = 3$
4	a (5)	edge (a, c) skipped, c is closed	closed, $g(c) = 3$
5	t (6)	returns (s, b, c, t) , cost 6	

Here c was closed before $g(c)$ was exact, and the closed set blocked the correction. A^* with reopening removes the condition “ v is not in S ” from Algorithm 4 and moves every vertex whose g decreases back from S into Q . On Example 1 it reopens c with key 3 and returns (s, a, c, t) of cost 5.

Theorem 2 (Optimality of A^* with an admissible heuristic). *Let G be a finite graph in which every edge weight is positive, let s and t be vertices of G such that t is reachable from s , and let h be an admissible heuristic. Then A^* with reopening terminates and returns a path from s to t of cost $\delta(s, t)$.*

Proof. We first prove termination. Let $w_{\min} > 0$ be the smallest edge weight, let v be a vertex, and let g_0 be the first finite value of $g(v)$. By Lemma 2, every later value of $g(v)$ is the cost of a walk from s to v of cost at most g_0 . Such a walk has at most $g_0/w_{\min}$ edges, so there are finitely many such walks, and $g(v)$ takes finitely many values. After an extraction, the vertex v returns to Q only if $g(v)$ decreases, so v is extracted at most once for each value of $g(v)$. Hence the number of iterations is finite.

Let $P^* = (s = v_0, v_1, \dots, v_k = t)$ be a shortest path from s to t . We show that before every extraction that precedes the extraction of t , some vertex v_j of P^* is in Q with $g(v_j) = \delta(s, v_j)$. In particular Q is not empty, so the algorithm does not return failure. Let j be the largest index with $g(v_j) = \delta(s, v_j)$; it exists because $g(s) = 0$. Suppose that v_j is not in Q . Since $g(v_j)$ is finite, v_j has been reached, so it is in S ; in particular $v_j \neq t$, because the extraction of t ends the search. The value $g(v_j)$ did not decrease after the last expansion of v_j , because a decrease would have reopened v_j . Hence $g(v_j) = \delta(s, v_j)$ during that expansion, which relaxed the edge (v_j, v_{j+1}) and set $g(v_{j+1}) \leq \delta(s, v_j) + w(v_j, v_{j+1}) = \delta(s, v_{j+1})$. By Lemma 2, $g(v_{j+1}) = \delta(s, v_{j+1})$ from then on, which contradicts the choice of j . So v_j is in Q .

When t is extracted, its key is $g(t) + h(t) = g(t)$, because admissibility gives $0 \leq h(t) \leq h^*(t) = 0$. Just before this extraction, the vertex v_j found above is in Q with key

$$\delta(s, v_j) + h(v_j) \leq \delta(s, v_j) + h^*(v_j) = C^*, \quad (8)$$

where the equality holds because v_j lies on a shortest path from s to t . EXTRACT-MIN chose t , so $g(t) \leq C^*$. Together with $g(t) \geq C^*$ from Lemma 2, this gives $g(t) = C^*$.

Finally, consider the returned path. For every vertex $v \neq s$ on it, the last change of $g(v)$ and $\pi(v)$ set $g(v) = g(\pi(v)) + w(\pi(v), v)$, and $g(\pi(v))$ can only have decreased since. Hence $g(\pi(v)) + w(\pi(v), v) \leq g(v)$. Every weight is positive, so g strictly decreases along the predecessors; the path therefore visits no vertex twice and ends at s . Adding the inequalities along the path shows that its cost is at most $g(t) - g(s) = C^*$. No path costs less than C^* , so the cost of the returned path is exactly C^* . $\square$

Theorem 2 is the original result of Hart, Nilsson and Raphael [1]; Theorem 1 covers the usual implementation [2], in which each vertex is expanded at most once. With an inconsistent heuristic a vertex may be expanded many times (Section 7).

7 Time and space complexity

We analyze Algorithm 4 with a consistent heuristic, where $|V|$ and $|E|$ are the numbers of vertices and edges. We assume that Q is a binary heap, in which each queue operation takes $O(\log |V|)$ time, and that $h(v)$ is computed in constant time, as for the distance heuristics.

7.1 Time

By Theorem 1, every vertex is extracted at most once, so every edge is examined at most twice (Table 17).

Table 17: Running time of Algorithm 4 with a binary heap.

Operation	Number of executions	Cost of each	Total
initialization of g and π	once per vertex	$O(1)$	$O(V)$
EXTRACT-MIN	at most once per vertex	$O(\log V)$	$O(V \log V)$
examination of an edge	at most twice per edge	$O(1)$	$O(E)$
INSERT-OR-DECREASE-KEY	at most once per examination of an edge	$O(\log V)$	$O(E \log V)$

Summing the totals gives the running time

$$O(|V|) + O(|V| \log |V|) + O(|E|) + O(|E| \log |V|) = O((|V| + |E|) \log |V|). \quad (9)$$

This is also the bound for Dijkstra’s algorithm, and a Fibonacci heap reduces both to $O(|E| + |V| \log |V|)$ [7]. In the worst case A* is no faster than Dijkstra’s algorithm, since the zero heuristic is consistent; its advantage lies in the number of vertices expanded (Section 8).

With an inconsistent heuristic, A* with reopening can expand a vertex many times; Martelli [8] gave graphs on which the number of expansions grows exponentially.

7.2 Space

Each item of Table 18 holds at most one entry per vertex, so the memory is $O(|V|)$.

Table 18: Memory used by Algorithm 4.

Data	Size	Total
values g and predecessors π	one entry per vertex	$O(V)$
closed set S	at most every vertex	$O(V)$
priority queue Q	at most one entry per vertex, with DECREASE-KEY	$O(V)$
reconstructed path	at most $ V $ vertices	$O(V)$
Total		$O(V)$

Implementations that insert a second queue entry instead of decreasing a key need $O(|V| + |E|)$ space. On large graphs memory, not time, is usually the limit, because A* stores every vertex it reaches [2].

7.3 Comparison of the three algorithms

In greedy best-first search every vertex enters Q once and no key decreases, which gives the smallest bound in Table 19. The worst-case bounds do not separate A* from Dijkstra’s algorithm.

Table 19: Worst-case complexity of the three algorithms with a binary heap and a consistent heuristic.

	Dijkstra's algorithm	Greedy best-first search	A* search
Time	$O((V + E) \log V)$	$O(E + V \log V)$	$O((V + E) \log V)$
Space	$O(V)$	$O(V)$	$O(V)$
Shortest path	guaranteed	not guaranteed	guaranteed

8 Why A* expands fewer vertices

Although the worst-case bounds coincide, A* expands far fewer vertices than Dijkstra's algorithm in practice. This section characterizes the vertices it expands and measures the effect on a grid with an obstacle.

8.1 The vertices that A* expands

Proposition 2. *Let G be a finite graph with nonnegative edge weights, let t be reachable from s , and let h be a consistent heuristic. Then Algorithm 4 has the following two properties.*

- (a) *Every vertex u that it extracts satisfies $\delta(s, u) + h(u) \leq C^*$.*
- (b) *Every vertex u with $\delta(s, u) + h(u) < C^*$ is extracted before t .*

Proof. (a) By Lemma 5, a vertex u is extracted with key $g(u) + h(u) = \delta(s, u) + h(u)$. The destination is extracted last, with key $\delta(s, t) + h(t) = C^*$, and by Lemma 4 no earlier key is larger.

(b) Suppose that such a vertex u is not in S when t is extracted. Let P be a shortest path from s to u , and let x be the first vertex of P that is not in S . The argument of the proof of Lemma 5 shows that x is in Q with $g(x) = \delta(s, x)$, and Lemma 3 gives $f(x) \leq \delta(s, u) + h(u) < C^*$. The key of t is C^* , so $x \neq t$, and EXTRACT-MIN would have returned a vertex with a key below C^* instead of t . This is a contradiction. $\square$

In words, A* expands only vertices whose best route is estimated to cost at most C^* . For the zero heuristic, part (b) says that Dijkstra's algorithm expands every v with $\delta(s, v) < C^*$ (Remark 1).

Corollary 2. *Let h be a consistent heuristic. Every vertex u that A* expands but Dijkstra's algorithm does not expand satisfies $\delta(s, u) = C^*$ and $h(u) = 0$.*

Proof. Let u be such a vertex. By part (a) of Proposition 2, $\delta(s, u) + h(u) \leq C^*$, and since $h(u) \geq 0$, we have $\delta(s, u) \leq C^*$. By part (b) applied to the zero heuristic, Dijkstra's algorithm expands every vertex v with $\delta(s, v) < C^*$, so $\delta(s, u) = C^*$. Then part (a) gives $h(u) \leq 0$, that is, $h(u) = 0$. $\square$

Apart from such borderline vertices, A* expands a subset of what Dijkstra's algorithm expands. The set $\{u : \delta(s, u) + h(u) \leq C^*\}$ shrinks as h grows, which justifies requirement (iii) of Section 5.1; for $h = h^*$ it contains only the vertices on shortest paths.

On the running example, $\delta(s, v) + h(v)$ is 7 for D, N, B, H, 8 for S, and at least 9 for the others, so A* expands exactly the five cities of Table 9.

Geometrically, with the zero heuristic this set is a disk around s ; with the Euclidean heuristic it is a thin ellipse around the segment from s to t (Figure 18).

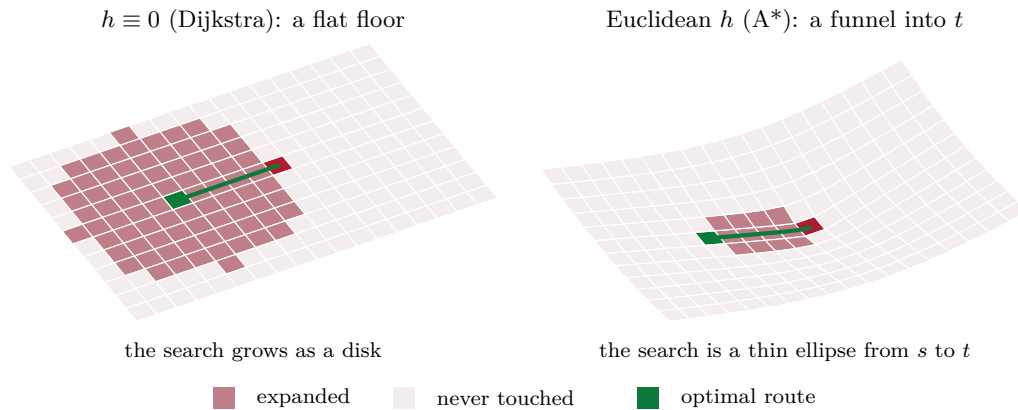


Figure 18: A schematic comparison on one grid. With the zero heuristic the expanded cells form a disk around s ; with the Euclidean heuristic they form a thin ellipse from s to t .

8.2 Experiment: a grid with a U-shaped wall

An obstacle tests how A^* copes with information that its heuristic does not contain. The instance is as follows (Figure 19).

Graph. The 139 free cells of a 16×10 grid; moves go up, down, left or right and cost 1.

Wall. A U of 21 cells opening toward the source: column 12, rows 3–7, and rows 3 and 7, columns 4–12.

Query. From $s = (1, 4)$ to $t = (14, 6)$, behind the wall, with the Manhattan distance as heuristic.

Ties are broken first in, first out. Table 20, Figure 20 and Figure 21 give the results.

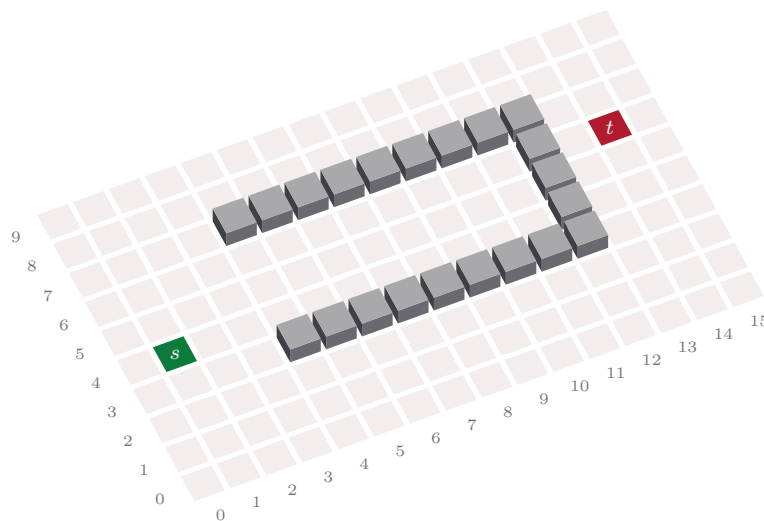


Figure 19: The 16×10 test grid. The raised blocks form the U-shaped wall; the source s is green and the destination t is red.

Dijkstra's algorithm expands almost the whole grid. Greedy best-first search fills the inside of the U and returns a path 16 moves longer than necessary.

A^* is also drawn into the U, because the Manhattan distance ignores the wall. Every cell inside the U is reached by moves toward the destination, so its key equals $h(s) = 15 < 19 = C^*$, and Proposition 2 forces A^* to expand it. A^* then goes around the wall and returns a shortest path, after 81 expansions, all among the 137 of Dijkstra's algorithm (Corollary 2).

Table 20: The three algorithms on the grid with the U-shaped wall.

	Dijkstra's algorithm	Greedy best-first	A* search
Cells expanded (of 139 free)	137	48	81
Path cost (moves)	19	35	19
Extra moves over the optimum	0	16	0
Shortest path	yes	no	yes

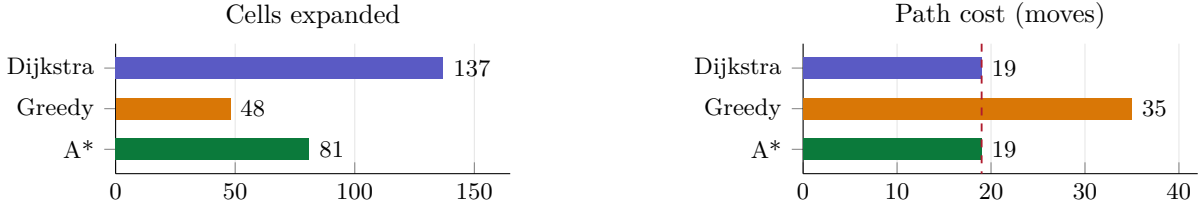
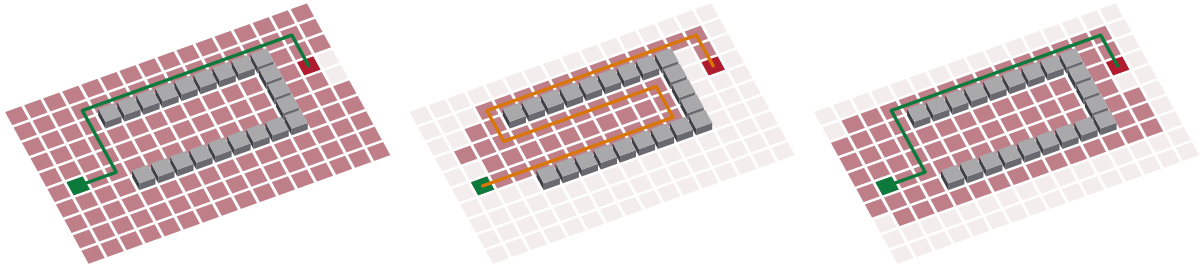


Figure 20: The results of Table 20 as bar charts. The dashed line marks the optimal cost.



(a) Dijkstra's algorithm: 137 cells expanded, path cost 19. (b) Greedy best-first search: 48 cells expanded, path cost 35. (c) A* search: 81 cells expanded, path cost 19.

Figure 21: The three algorithms on the grid. Dark tiles were expanded, and the line is the returned path.

Figure 22 raises every cell to the step at which it was expanded: Dijkstra's algorithm spreads evenly from s , whereas A* first fills the U and then climbs a narrow staircase around the wall.

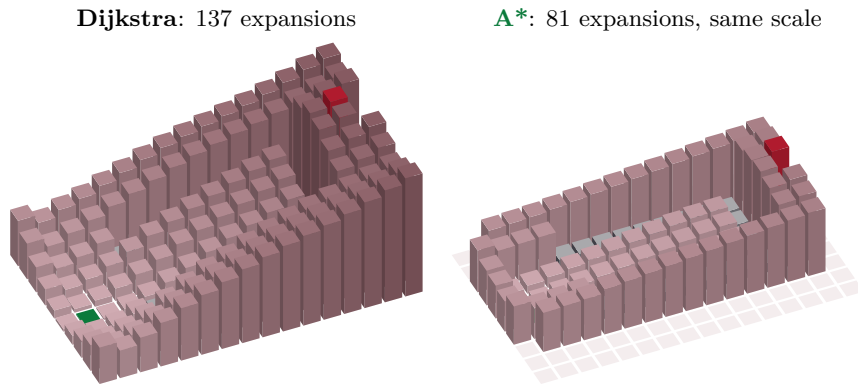


Figure 22: Every expanded cell raised to the step at which it was expanded, on the same scale in both panels.

9 Worked example on a real map: Dhaka to Sylhet

We now follow the three algorithms from Dhaka to Sylhet on a map of Bangladesh.

9.1 The district graph

The instance is as follows (Figure 23).

Graph. The 64 districts of Bangladesh, placed at their centres, with 133 edges between neighboring districts.

Weights. The great-circle distance $d(u, v)$ between district centres, that is, the shortest distance along the Earth's surface, computed with the haversine formula.²

The heuristic is the straight-line distance to Sylhet,

$$h(v) = d(v, \text{Sylhet}), \quad (10)$$

which is consistent because the weights are themselves straight-line distances (Table 10).



Figure 23: The district graph: 64 districts of Bangladesh joined by 133 edges between neighbors. The source, Dhaka, is green, and the destination, Sylhet, is red.

9.2 Dijkstra's algorithm

Dijkstra's algorithm returns the optimal 215.1 km route but expands 50 of the 64 districts (Figure 24, Table 21). Jashore and Naogaon lie away from Sylhet but are expanded first because they are closer to Dhaka (Remark 1).

²For two points with latitudes φ_1, φ_2 and longitudes λ_1, λ_2 on a sphere of radius R , the haversine formula gives the distance $d = 2R \arcsin \sqrt{\sin^2((\varphi_2 - \varphi_1)/2) + \cos \varphi_1 \cos \varphi_2 \sin^2((\lambda_2 - \lambda_1)/2)}$; we use $R = 6371$ km.

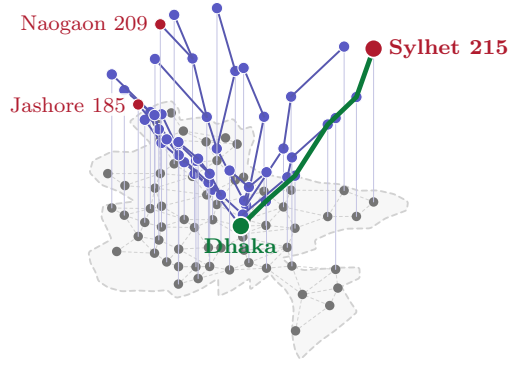


Figure 24: The search tree of Dijkstra's algorithm from Dhaka, with every expanded district lifted to its cost g in kilometres. The tree grows in every direction; Jashore and Naogaon lie away from Sylhet but are expanded before it.

Table 21: Dijkstra's algorithm from Dhaka to Sylhet.

Districts expanded	50 of 64
Districts on the route	6
Route	Dhaka, Narsingdi, Brahmanbaria, Habiganj, Moulvibazar, Sylhet
Route cost	215.1 km (optimal)

9.3 Greedy best-first search

Greedy best-first search rolls down the funnel of h (Figure 25). It expands only six districts, but its route is 28.1 km (13%) longer than the optimum.

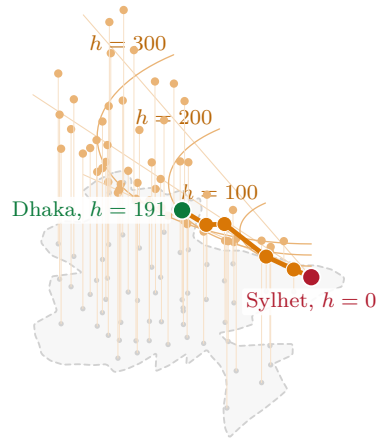


Figure 25: Every district lifted to h , its straight-line distance to Sylhet in kilometres. Greedy best-first search (orange) always moves to the lowest district it has seen.

The wrong turn is at Narsingdi (Figure 26): Kishoreganj looks 8.4 km closer to Sylhet, but the road to it is 18.2 km longer. The key of A* prefers Brahmanbaria (203.5 km against 213.3 km). The two routes differ in this one district (Figure 27).

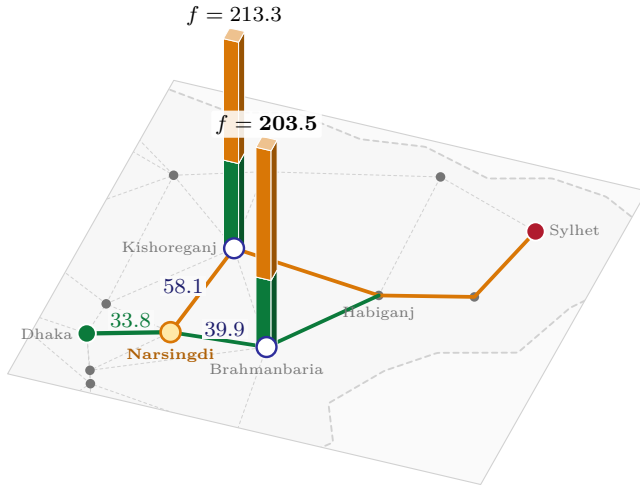


Table 22: The two roads out of Narsingdi, where $g = 33.8$ km. All values are in kilometres.

	Kishoreganj	Brahmanbaria
road from Narsingdi	58.1	39.9
h to Sylhet	121.4	129.8
g through Narsingdi	91.9	73.7
$f = g + h$	213.3	203.5
chosen by greedy search	✓	
smaller key f (A^*)		✓

Figure 26: The choice at Narsingdi. Each column shows g (green) with h (orange) on top, so its height is f .

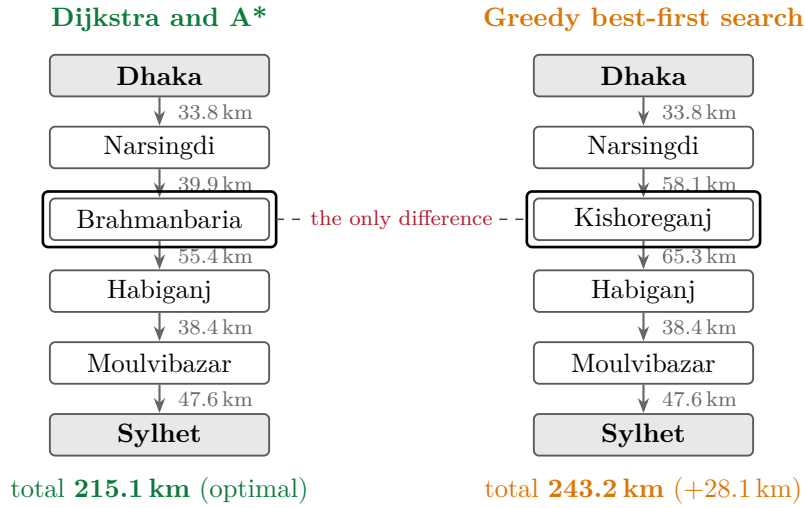


Figure 27: The routes from Dhaka to Sylhet as flow charts, with the length of every leg.

9.4 A* search

A* returns the optimal route after eight expansions (Figure 28, Table 23). The run illustrates three results of the report:

- (1) The off-route districts Gazipur and Kishoreganj are expanded because their keys are below C^* (Proposition 2).
- (2) Relaxation repairs the detour: Habiganj is first reached through Kishoreganj at 147.3 km and then through Brahmanbaria at 129.1 km.
- (3) The keys never decrease (Lemma 4).

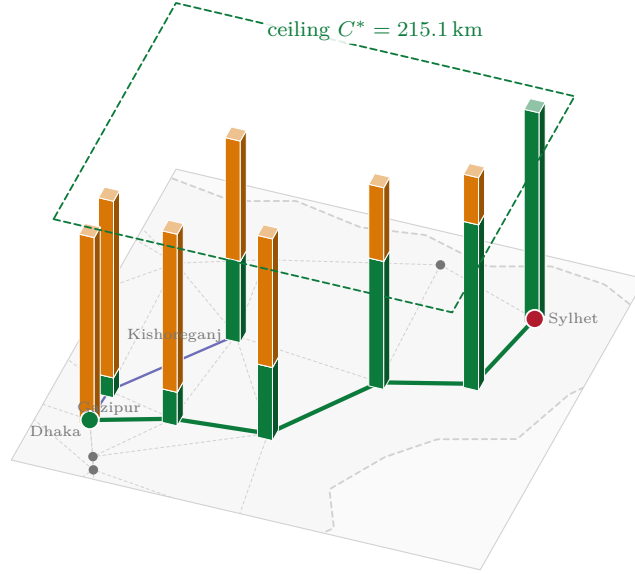


Figure 28: The eight districts expanded by A*, as columns of g (green) and h (orange). No column rises above the optimal cost $C^* = 215.1$ km (dashed).

Table 23: The districts expanded by A* from Dhaka to Sylhet, in order. All values are in kilometres.

Step	District	g	h	$f = g + h$
1	Dhaka	0.0	191.2	191.2
2	Narsingdi	33.8	159.5	193.3
3	Gazipur	20.0	178.3	198.3
4	Kishoreganj	82.0	121.4	203.4
5	Brahmanbaria	73.7	129.8	203.5
6	Habiganj	129.1	74.4	203.5
7	Moulvibazar	167.5	47.6	215.1
8	Sylhet	215.1	0.0	215.1

9.5 Comparison

Table 24, Figure 29 and Figure 30 compare the three algorithms.

Table 24: The three algorithms on the district graph, from Dhaka to Sylhet.

	Dijkstra's algorithm	Greedy best-first	A* search
Districts expanded (of 64)	50	6	8
Route cost (km)	215.1	243.2	215.1
Extra cost over the optimum	0	28.1 km (13%)	0
Shortest route	yes	no	yes

Figure 31 explains the gap. Flooding the map to C^* submerges the 50 districts with $g \leq C^*$, which Dijkstra's algorithm expands, but only the eight with $g + h \leq C^*$, which A* expands (Proposition 2).

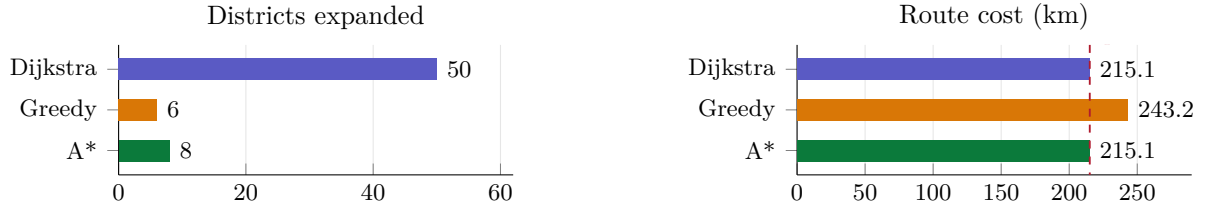


Figure 29: The results of Table 24 as bar charts. The dashed line marks the optimal cost.

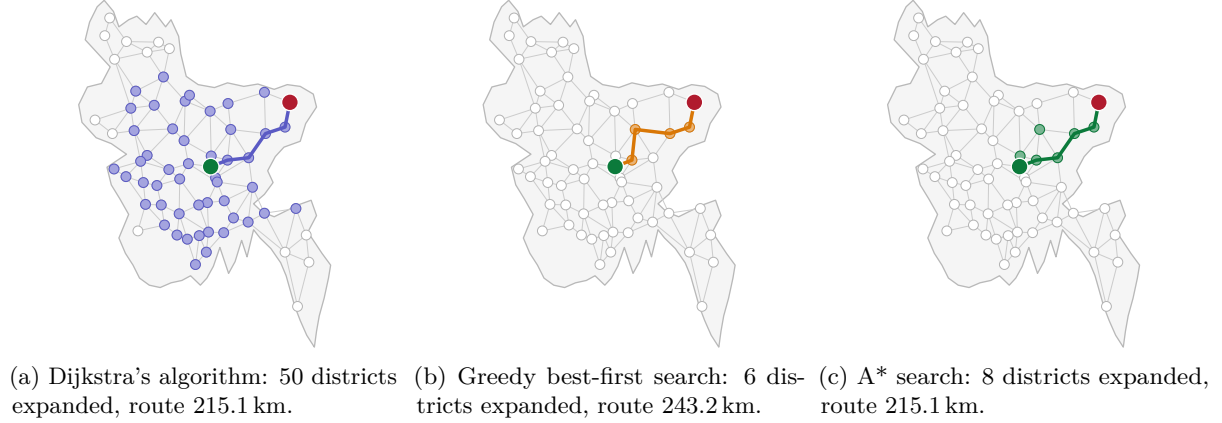


Figure 30: The districts expanded (filled) and the route returned by each algorithm from Dhaka (green) to Sylhet (red).

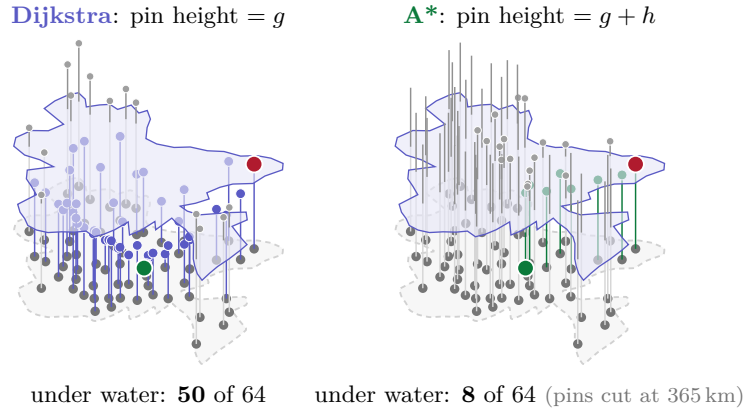


Figure 31: Every district as a pin, flooded to $C^* = 215.1$ km. Left: pin height g . Right: pin height $g + h$. Adding h lifts every district that lies away from Sylhet out of the water.

9.6 Summary of the experiments

Table 25 collects all results. A* always returned a shortest path, with 38%, 41% and 84% fewer expansions than Dijkstra's algorithm on the three instances. Greedy best-first search expanded the fewest vertices but never returned a shortest path.

Table 25: Vertices expanded and cost of the returned path on the three instances.

	Running example		Grid		District graph	
	expanded	cost	expanded	cost	expanded	cost
Dijkstra’s algorithm	8	8	137	19	50	215.1 km
Greedy best-first search	4	10	48	35	6	243.2 km
A* search	5	8	81	19	8	215.1 km

10 Variants of A*

Table 26 lists four variants of A* that address its limitations: the dependence on the heuristic, the memory needed, and the cost of searching from scratch after every change of the map.

Table 26: Four variants of A* and the limitation that each one addresses.

Variant	Idea	Limitation addressed
Weighted A* [9]	rank by $g(v) + \varepsilon h(v)$ with $\varepsilon > 1$	too many expansions; with an admissible h , the returned path costs at most εC^*
IDA* [3]	a series of searches that each follow one path at a time as far as possible (depth-first search), with a growing limit on f	memory: only the current path is stored, at the price of repeated expansions
Jump point search [10]	skip the many paths of equal cost on uniform grids with diagonal moves	many equivalent expansions on grids; the path remains optimal
D* Lite [11]	repair the previous search when the cost of an edge changes	maps that change while a robot or a game character moves

11 Conclusion

A* meets both requirements of Section 1.4. The cost so far g keeps it optimal, as in Dijkstra’s algorithm, and the estimate h directs it toward the destination, as in greedy best-first search.

With a consistent heuristic, A* expands every vertex at most once and returns a shortest path (Theorem 1). With an admissible but inconsistent heuristic, a shortest path is guaranteed only when closed vertices can be reopened (Theorem 2), and with an overestimating heuristic no guarantee remains. The worst-case cost equals that of Dijkstra’s algorithm, but A* expands only vertices with $\delta(s, u) + h(u) \leq C^*$ (Proposition 2). In our experiments it expanded 81 cells against 137 on the grid and eight districts against 50 on the map of Bangladesh, always returning a shortest path.

Since a larger admissible heuristic leaves fewer vertices to expand, choosing a heuristic that is cheap and as close as possible to the true cost is the main design decision when A* is applied to a new problem.

Code availability. The programs that produce every trace, table and experimental result of this report are available at <https://github.com/ishmamzarif/a-star-report>.

References

- [1] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- [2] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- [3] Richard E. Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1):97–109, 1985. doi: 10.1016/0004-3702(85)90084-0.
- [4] Richard E. Korf. Finding optimal solutions to Rubik’s Cube using pattern databases. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence*, pages 700–705, 1997.
- [5] Baeldung. Priority queue. <https://www.baeldung.com/cs/priority-queue>. Accessed 1 October 2026.
- [6] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.
- [7] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.
- [8] Alberto Martelli. On the complexity of admissible search algorithms. *Artificial Intelligence*, 8(1):1–13, 1977. doi: 10.1016/0004-3702(77)90002-9.
- [9] Ira Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4): 193–204, 1970. doi: 10.1016/0004-3702(70)90007-X.
- [10] Daniel Harabor and Alban Grastien. Online graph pruning for pathfinding on grid maps. In *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, pages 1114–1119, 2011.
- [11] Sven Koenig and Maxim Likhachev. D* Lite. In *Proceedings of the Eighteenth National Conference on Artificial Intelligence*, pages 476–483, 2002.